

PatchEmbed3DPlus1D — Patchify architecture for fMRI

2026-06-14

Overview

The patchify module `PatchEmbed3DPlus1D` is the fMRI-specific input encoder that turns a 4D scan (T, X, Y, Z) into a flat sequence of ($T_{\text{eff}} \times N_{\text{spatial}}$) tokens consumed by the ViT.

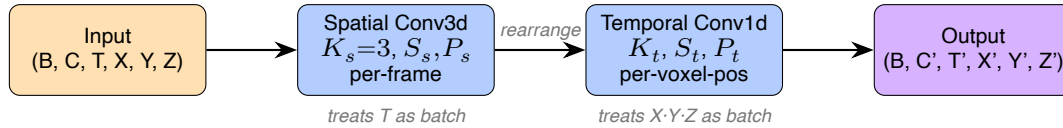
It is a **hierarchical encoder inspired by the MovieGen Temporal Auto-Encoder (TAE)** (github.com/MathieuTuli/MovieGen, `tae.py:740`), adapted to 4D fMRI.

File: `dinov2/layers/patch_embed_3d_plus_1d.py` — 4 classes :

Line	Class	Role
29	<code>Conv3Plus1d</code>	Separable spatial-3D + temporal-1D conv (basic block of every level)
68	<code>_ResBlock3Plus1d</code>	Residual block stacking $2 \times$ <code>Conv3Plus1d</code>
85	<code>PositionEmbedding3D</code>	Factorised positional embedding (temporal + spatial + CLS)
117	<code>PatchEmbed3DPlus1D</code>	Top-level encoder stacking the 3 hierarchical levels

1. The basic building block: Conv3Plus1d

A **separable 4D convolution** = 3D spatial conv (per frame) + 1D temporal conv (per spatial position).



Why “Plus” instead of full 4D conv

PyTorch has no native Conv4d. A full (T, X, Y, Z) kernel of size K^4 would have $K^4 \cdot C^2$ parameters. The separable version $\text{Conv3d}(K^3) + \text{Conv1d}(K)$ has only $K^3 \cdot C^2 + K \cdot C^2 = (K^3 + K) \cdot C^2$ parameters — about K **times fewer**.

This is the same trick as MovieGen’s Conv2Plus1d (used in video) extended to 3D spatial.

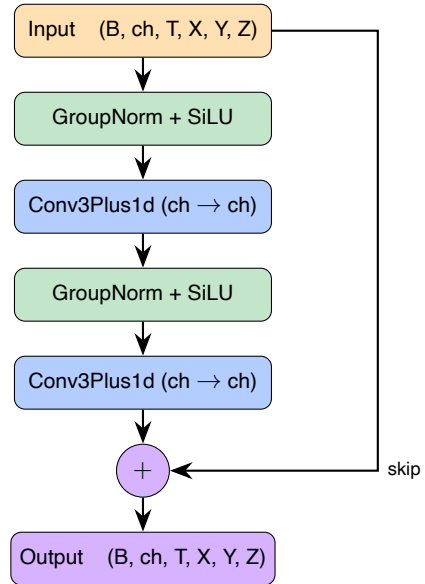
Equivalent PyTorch primitive

```
class Conv3Plus1d(nn.Module):
    def __init__(self, in_c, out_c, K_s=3, S_s=1, P_s=1,
                  K_t=3, S_t=1, P_t=1):
        super().__init__()
        self.spatial = nn.Conv3d(in_c, out_c, K_s, S_s, P_s)
        self.temporal = nn.Conv1d(out_c, out_c, K_t, S_t, P_t)

    def forward(self, x):  # x: (B, C, T, X, Y, Z)
        x = rearrange(x, 'b c t x y z -> (b t) c x y z')
        x = self.spatial(x)  # 3D conv per frame
        x = rearrange(x, '(b t) c x y z -> (b x y z) c t', b=B, t=T)
        x = self.temporal(x)  # 1D conv per voxel-pos
        x = rearrange(x, '(b x y z) c t -> b c t x y z', b=B, x=X, y=Y, z=Z)
        return x
```

2. The residual block: `_ResBlock3Plus1d`

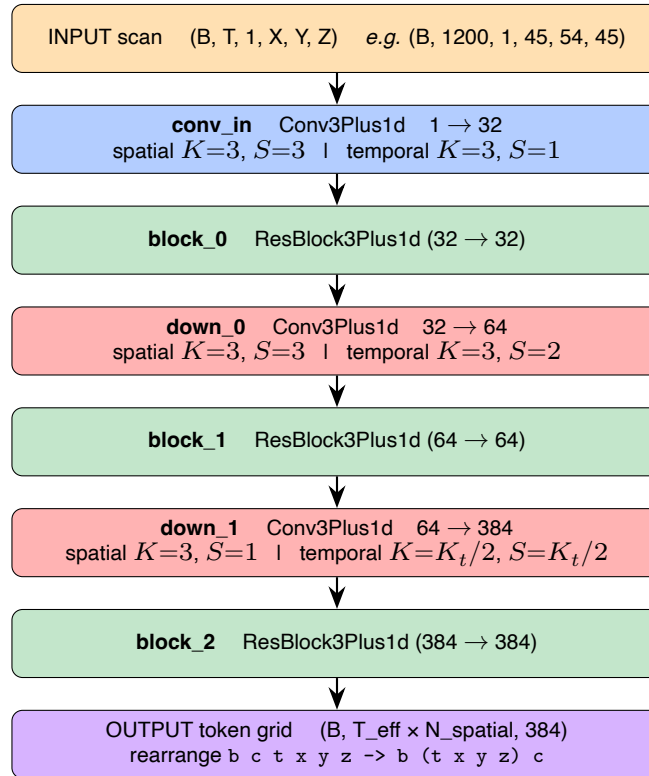
Two `Conv3Plus1d` stacked with `GroupNorm + SiLU` + residual connection.



Each `ResBlock` keeps the **same shape** (channels and spatial/temporal dimensions). Equivalent to MovieGen's `TemporalResnetBlock` (`tae.py:643`).

3. Full encoder: PatchEmbed3DPlus1D

Three hierarchical levels chained with downsampling between them.



4. Shape evolution — concrete examples

HCP-YA setup ($T = 1200$, $\text{temporal_kernel} = 20$)

Stage	Operation	Output shape (B, C, T, X, Y, Z)	Comment
input	—	(B, 1, 1200, 45, 54, 45)	raw 4D scan
conv_in	spatial /3	(B, 32, 1200, 15, 18, 15)	first spatial downsample
block_0	identity	(B, 32, 1200, 15, 18, 15)	residual
down_0	spatial /3, T /2	(B, 64, 600, 5, 6, 5)	total /9 spatial, /2 temporal
block_1	identity	(B, 64, 600, 5, 6, 5)	residual
down_1	T /10	(B, 384, 60, 5, 6, 5)	total /9 spatial, /20 temporal
block_2	identity	(B, 384, 60, 5, 6, 5)	residual
tokens	rearrange	(B, 60 × 150, 384) = (B, 9000, 384)	$T_{\text{eff}} \times N_{\text{spatial}} \times \text{embed_dim}$

Mixed-T140 setup ($T = 140$, $\text{temporal_kernel} = 14$)

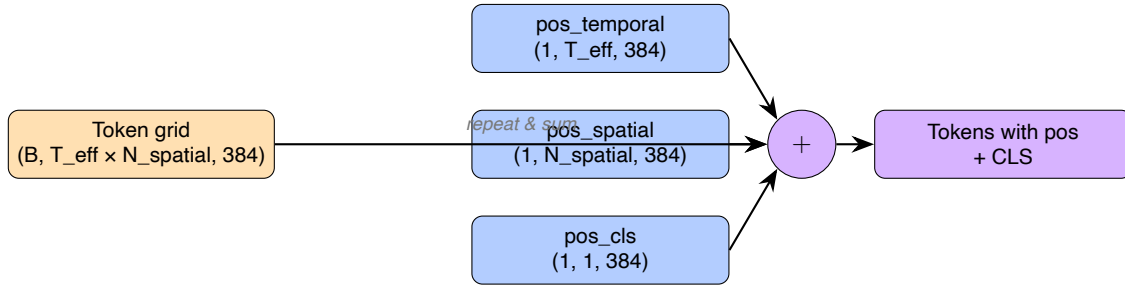
Stage	Operation	Output shape	Comment
input	—	(B, 1, 140, 45, 54, 45)	raw 4D scan
conv_in	spatial /3	(B, 32, 140, 15, 18, 15)	
block_0	identity	(B, 32, 140, 15, 18, 15)	
down_0	spatial /3, T /2	(B, 64, 70, 5, 6, 5)	
block_1	identity	(B, 64, 70, 5, 6, 5)	
down_1	T /7	(B, 384, 10, 5, 6, 5)	$K_t = 14/2 = 7$
block_2	identity	(B, 384, 10, 5, 6, 5)	
tokens	rearrange	(B, 10 × 150, 384) = (B, 1500, 384)	6× fewer tokens than HCP-YA

Effective strides

- **Spatial total stride** = $3 \times 3 \times 1 = 9$ □ matches `patch_size = 9`
- **Temporal total stride** = $1 \times 2 \times K_t/2 = K_t = \text{temporal_kernel}$

5. Factorised positional embedding (PositionEmbedding3D)

After the encoder, we add a **factorised** positional embedding before the ViT — three small tables broadcast and summed.



Parameter count comparison (HCP setup)

Encoding	Shape	Params
Flat 2D pos table (vanilla ViT)	$(1, T_{\text{eff}} \times N_{\text{spatial}}, 384) = (1, 9000, 384)$	3.46 M
Factorised (ours)	pos_temporal (60×384) + pos_spatial (150×384) + pos_cls (1×384)	80 K

Factor of **~43x fewer parameters** for the positional embedding.

Combine method (per scan)

```
def combined_patch_pos(self):
    pos_t = repeat(self.pos_temporal, '1 t d -> 1 (t n) d', n=N_spatial)
    pos_s = repeat(self.pos_spatial, '1 n d -> 1 (t n) d', t=T_eff)
    return pos_t + pos_s # (1, T_eff*N_spatial, embed_dim)
```

The CLS token gets pos_cls added separately when prepended in prepare_tokens_with_masks of the ViT.

6. Memory and compute notes

Activation checkpointing

The full hierarchical encoder produces large intermediate activations (the level-0 tensor at full spatial resolution is $(B, 32, 1200, 15, 18, 15) = \sim 16$ GB for $B=2$ in fp32). We wrap the encoder forward in `torch.utils.checkpoint`:

```
if self.training:
    x = torch.utils.checkpoint.checkpoint(
        self._encoder_forward, x, use_reentrant=False,
    )
else:
    x = self._encoder_forward(x)
```

Trade-off: **~30% extra forward compute** (re-computation during backward) for **~80 GB saved VRAM**.

Parameter count

Module	Parameters
conv_in	~870
block_0	~37 k
down_0	~55 k
block_1	~147 k
down_1 (Kt=14)	~840 k
block_2	~5.3 M
PositionEmbedding3D	~80 k
Total	~6.5 M

(vs ~21 M for the 12 ViT-S transformer blocks downstream)

7. Why this design

Choice	Alternative	Why this one
Separable Conv3+1D	Full 4D conv	PyTorch has no Conv4d; separable has $\sim K$ times fewer params
3 hierarchical levels with downsample	Single big patchify conv	Pyramidal feature hierarchy (à la TAE / Hiera) captures multi-scale structure
Conv3Plus1d in every level	Spatial-only then temporal-only split	Avoids the “spatial collapse before temporal context” issue
GroupNorm + SiLU	BatchNorm + ReLU	BatchNorm needs batch stats, problematic with small batches / FSDP wrapping
Factorised pos embed	Flat 2D pos embed	$O(T_{\text{eff}} + N_{\text{spatial}})$ params instead of $O(T_{\text{eff}} \times N_{\text{spatial}})$
temporal_kernel derived from config	Hard-coded down_1 K_t	Lets us train at $K_t = 20$ (HCP) and $K_t = 14$ (mixed) from same code
Activation checkpointing on encoder	No checkpointing	Encoder produces the largest intermediate tensors; ViT downstream is smaller

Reference

- **Original architecture inspiration:** MovieGen Temporal Auto-Encoder, `tae.py` lines 740 (TemporalEncoder), 573 (Conv2Plus1d), 643 (TemporalResnetBlock). See github.com/MathieuTuli/MovieGen.
- **Code in this repo:** `dinov2/layers/patch_embed_3d_plus_1d.py`
- **Integration point:** `dinov2/models/__init__.py:build_model_from_cfg` injects PatchEmbed3DPlus1D as the `embed_layer` of `DinoVisionTransformer` when `cfg.student.fmri_mode = true`.